2026 年 9 月 17 日 · 对外经济贸易大学

AI 时代的研究范式：从模型能力到工作流设计

基础版 · 与 AI 协作的方法与个人经验

陶学臻

上海财经大学商学院

开场

AI 能完成许多任务，但仍需要指导

即使输入不多，AI 也能完成相当多的工作，如生成幻灯片、起草文档、编写代码。



但直接产出与审查后的结果之间仍有差距，主要来自三个方面：

容易出错

数据分析、文档处理、资料查找中时有错误，还会产生幻觉。

产出千篇一律

幻灯片风格高度雷同，语言表述与个人风格相去甚远。

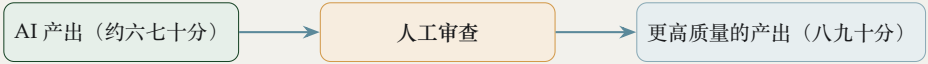
复杂任务不够准确

步骤较多的任务，没有额外提示时，结果往往不够准确。

因此，需要找到合适的方法，**保证并尽可能提高 AI 的产出质量**。

人工审查做什么，本讲交流什么

在 AI 产出六七十分的基础上，进一步的提升主要依靠人工审查。



人工审查的两项工作

1. **修正错误**：纠正 AI 自身产生的错误，如幻觉和常见的偶发错误
2. **补充背景**：持续补充信息，让 AI 更了解任务需求、个人风格和其他相关情况

本讲交流的重点：审查的形式与流程

1. **交互高效**：人如何快速阅读 AI 的产出，又如何把意见高效地反馈给 AI
2. **减轻负担**：让 AI 更好地理解个人需求，减少人需要修改和审查的工作

后面几部分结合具体做法，围绕这两个方面展开。

本讲内容

- **一、与模型交流**　提示词的写法，以及上下文与规则的设置
- **二、模型与工具**　模型的选择、智能体的运行框架（harness），以及 AI IDE 的使用
- **三、我的一些个人经验：做法与工具**　修改文稿、反馈意见和积累资料的做法，以及用 AI IDE 搭建的工具
- **四、在研究中的应用**　用大语言模型从文本中构造变量，并检验其可靠性

1

与模型交流

提示词的写法，以及上下文与规则的设置

1. 提示词的写法
2. 零样本与少样本
3. 审查输出
4. 上下文与规则

提示词的三个要素：任务、材料与输出格式

任务

需要 AI 完成的任务

- 「清洗这份数据」
- 「写一段引言」
- 「给这段代码加注释」

材料

提供给 AI 的材料

- 数据文件、代码、论文段落
- 变量定义、样本范围
- 已有的运行结果

输出格式

期望的结果形式

- Stata .do 文件
- 三线表 / outreg2 格式
- 中文、300 字以内

需求表述：模糊与明确的对比

表述模糊

「帮我做个回归」

AI 不知道使用什么数据
AI 不知道聚类到哪一层
AI 不知道输出成什么格式
AI 不知道对照组如何选择

结果：AI 一旦理解有误，就需要重新来过

表述明确

「panel data，对 firm 聚类，输出 outreg2 表，对照组用 EV 渗透率 $<5\%$ 的城市」

数据格式已说明
聚类层级已确定
输出格式已确定
对照组定义已明确

结果：AI 可以直接给出能运行的代码

多花半分钟写清需求，可以省去 AI 数分钟的无效尝试。

提示：需求越具体，打字越难跟上思路，可以借助**语音输入法**口述，再交由 AI 整理。

CAREful 框架：提示词的四项检查

Context
背景和前提
所做研究的情况

Ask
具体要求
需要 AI 完成的事项

Rules
约束和限制
聚类、格式、语言

Examples
样例
期望输出的形式

提交前可对照这四项检查：缺少哪一项，AI 就会在该项上自行推测，而推测未必准确。

提示词实例：数据清洗

Prompt

`@data/raw_panel.csv`

这是一份城市–年份面板数据（2015–2023），请清洗：

1. 删除 GDP 或人口缺失的行
2. 标记出现少于 5 年的城市
3. 对 `GDP_per_capita` 在 1% 和 99% 处缩尾
4. 生成 `GDP_per_capita` 的对数变换
5. 将清洗后的数据保存到 `data/clean_panel.csv`
6. 输出所有变量的描述统计（N、均值、标准差、最小值、最大值）

写成一个 Stata `.do` 文件，每一步用注释说明。

零样本提示：直接给出指令，不提供示范

零样本（Zero-shot）指不提供示例，直接说明任务并由 AI 完成。这是最常用的方式，多数日常任务用一句话即可说明。

适用的情形

- 任务常见，AI 在训练语料中大量接触过（翻译、润色、编写代码、解释概念）
- 对输出格式没有严格统一的要求

不足的情形

- 需要统一口径 / 固定格式的批量任务（为上千条评论打分、抽取字段等）
- 判定标准较主观、边界模糊，AI 的判断容易前后不一致

此时可以改用**少样本**：提供几个「输入 → 输出」示范，把要求和判定口径固定下来。

用示范代替描述：修改文字风格

修改文字风格时，「正式一些」「更有学术性」这类描述较为抽象，AI 的理解未必与作者一致。给出一两段改写示范，AI 可以直接看到需要的样式。

只描述要求

提示词：「请把下面这段话改得正式一些。」

修改结果取决于 AI 对「正式」的理解：可能过于书面化、加入原文没有的套话，也可能改变原意。

提供示范

示范 1 改前：「这个结果说明补贴其实没啥用。」
改后：「结果表明，补贴的效果并不显著。」

示范 2 改前：「我们用双重差分比了一下政策前后。」
改后：「本文采用双重差分法比较政策前后的变化。」

待修改 「有些公司数据缺得厉害，我们就直接删了。」
请按上面示范的方式修改。

示范能把较复杂的要求具体化，用词、句子长短和语气都可以从中看出。风格要求越难用文字说清，这种方式的效果越明显。

少样本提示：提供示范，让 AI 按示范处理

零样本是直接给出指令；少样本是先提供 1–3 个“输入 → 输出”示范，再让 AI 按示范处理后续内容。任务对格式和口径有统一要求时（如信息抽取），这种方式尤其有效。

示范：按“购买意愿”对用户评论分档

示范 1 输入：“这车我已经订了，就等提车。” → 购买意愿 = **强**（已下单）
示范 2 输入：“车不错，但价格我还想再等等。” → 购买意愿 = **中**（有意向但观望）
示范 3 输入：“随便看看，这类车暂时不考虑。” → 购买意愿 = **弱**（无意向）

待处理 输入：“智驾确实炫，但担心这家新势力几年后倒闭没人修车。” → 请按上面格式给出购买意愿和理由。

三个示范使 AI 把握“强/中/弱”的口径，从而按同一标准处理成百上千条评论；这比用文字反复描述标准更可靠。

少样本实例：从财报文字构造“数字化转型指数”

场景：根据公司年报中的 MD&A 文字，给出 0–10 分的**数字化转型**评分。把评分口径写进示范后，AI 可以按一致的标准为上千份财报打分。

少样本提示词（可直接套用）

任务：读一段财报文字，按数字化转型程度打 0–10 分，并列出命中的关键词。
口径：明确提到**上云/中台/数据驱动/数字化/智能制造/AI 客服**等表述，命中越多、越具体，分越高；只字未提记 0 分。

示范 1 输入：“持续推进业务上云，搭建数据中台，以数据驱动运营，并上线 AI 客服。” → **9** 分；命中：上云、中台、数据驱动、AI 客服（多项具体举措）。
示范 2 输入：“启动数字化转型，部分核心业务已迁移上云，仍在推进中。” → **5** 分；命中：数字化、上云（有举措但较笼统）。
示范 3 输入：“报告期内加强信息化建设，上线一套内部管理系统。” → **3** 分；命中：信息化（未涉及云 / 中台 / AI）。
示范 4 输入：“主营钢材贸易，产销稳定，未涉及相关技术投入。” → **1** 分；命中：（无）。

待处理 输入：“（此处粘贴新财报片段）” → 请按同一口径给出分数与命中关键词。

CLEAR：审查 AI 输出的五个检查点

撰写提示词时参照 CAREful；得到回答后用 CLEAR 逐项核对。CLEAR 原为 Lo（2023）提出的提示词框架，这里用于审查输出。

Concise	**L**ogical	**E**xplicit	**A**daptive	**R**eliable
简明	逻辑严密	符合要求	持续改进	可靠
是否有空泛表述?	因果与推理是否成立?	是否符合格式与口径?	能否据此改进提示词?	数据/引用可信? 有无编造?

上下文与规则：由手动设置转为工具处理

这是提示词工程时期留下的两个概念。它们仍然适用，但现在大部分工作已由工具自动完成。

上下文：模型能看到的内容

2025 年，手动：@ 引用文件或文件夹 · 粘贴段落 · 避免对话过长

规则：长期有效的要求

2025 年，手动：在项目根目录放置规则文件

`RULE.md` · `CLAUDE.md` · `.cursor/rules/` · `AGENTS.md`

2026 年：两项工作由 agent 承担

自行检索项目文件 · 需要时读取 · 百万 token 上下文窗口 · 自动维护记忆文件 · 主动读取规则文件

仍需由人完成的工作

提供充分的材料 · 说明哪些内容不能改动 · 逐条写明严格的抽取标准（第四部分）

这两项工作并未消失，而是由人工操作转为工具自动处理；第二部分将介绍这类工具。

2

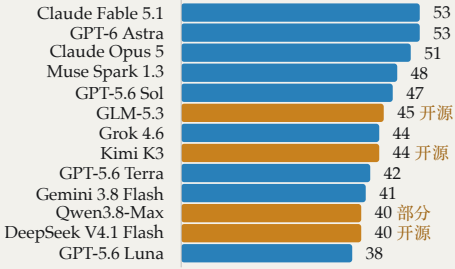
模型与工具

模型的选择与 AI IDE 的使用

1. 模型的选择
2. harness 与 loop
3. 认识 AI IDE
4. 两类 AI IDE
5. 工作目录与权限
6. 本地模型与微调

模型的选择：2026 年 9 月的格局

第一部分讨论如何表述需求，这一部分讨论模型与工具的选择。下图为同一天的综合评分。



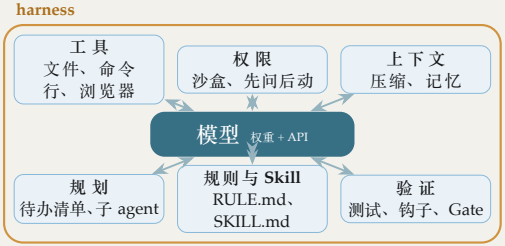
Artificial Analysis 智能指数 v4.3，2026 年 9 月 15 日。蓝色为美国实验室，金色为中国实验室；「开源」指权重已公开。

模型	发布	美元/百万 token 进/出
Claude Fable 5.1	9 月 1 日	10 / 50
GPT-6 Astra	9 月 3 日	10 / 50
Claude Opus 5	7 月 24 日	5 / 25
Muse Spark 1.3	9 月 2 日	1.25 / 4.25
GPT-5.6 Sol	7 月 9 日	4 / 20
GPT-5.6 Terra / Luna	7 月 9 日	2 / 12 · 0.2 / 1.2
Grok 4.6	8 月 12 日	2 / 6
Gemini 3.8 Flash	9 月 2 日	0.75 / 3.75
Kimi K3	7 月 16 日	3 / 15
GLM-5.3	8 月 14 日	1.4 / 4.4
Qwen3.8-Max	8 月 3 日	2 / 6
DeepSeek V4.1 Flash	9 月 10 日	0.15–0.3 / 0.6–1.2

国产开源模型比头部模型低八到十几分，价格仅为三分之一到几十分之一，可以用于重复性高、工作量大的任务。模型的选择取决于任务的性质。

模型与 agent：harness 的作用

模型本身只生成回答，不执行操作；使其能够执行操作的一整套组件称为 **harness**，即 Agent = 模型 + harness。第一部分的上下文与规则，现在都由这一层管理。

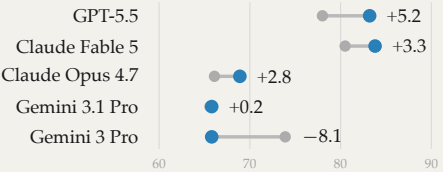


厂商	自有 harness	起始
Anthropic	Claude Code	2025-02
OpenAI	Codex	2025-04
Google	Antigravity CLI	2025-06
xAI	Grok Build	2026-05
月之暗面	Kimi Code CLI	2025-10
DeepSeek	DeepSeek Harness	2026-08
阿里	Qwen Code；Qoder	2025-07
字节	Trae	2025-01

早期测试者被问到「模型与 Claude Code 只能保留其一，保留哪个」时，选择的是 harness。另有一类不限定模型的 harness（Cursor、Droid、OpenCode、Cline），可以接入各家模型。

同一模型搭配不同 harness 的表现差异

Terminal-Bench 对每一组模型 × *harness* 分别计分。灰点为只提供一种工具的中性 harness，蓝点为厂商自有的 harness，模型与任务相同。



Terminal-Bench 2.1，2026 年 7 月由基准维护方统一运行，完成任务的百分比。蓝点为 Codex、Claude Code、Gemini CLI，灰点为 Terminus 2；Claude Fable 5 的两次运行推理强度不同。

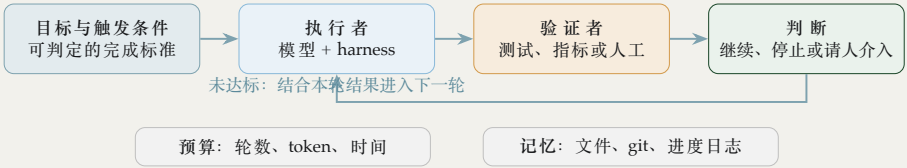
- 统一条件下，同一模型更换 harness 的差距为 **0.2** 至 **8** 分
- 厂商自有的 harness 并非总是领先：Gemini 3 Pro 在中性 harness 上高 **8** 分
- 第三方自行提交的成绩差距更大（最高约 **19** 分），但多数未经基准方核验

聊天框与 agent 的差别很大，前者无法执行命令；不同 agent 之间，统一条件下的差距多在几分以内。选择工具时，可以重点考察权限、规则、记忆与验证机制。

Loop 工程：harness 之上的循环机制

目前常见的一种层次划分：

prompt = 如何提问 → **context** = 掌握的信息 → **harness** = 能执行的操作 → **loop** = 如何持续运行



「我已经不再直接给 Claude 写提示词，而是运行一组循环，由这些循环给 Claude 写提示词。」（Boris Cherny，Claude Code 负责人，2026 年 6 月）最简单的 loop 是一个 bash 循环：反复输入同一条指令，以未通过的测试引导下一轮。

Loop 的效果：证据、发展阶段与风险

目前的证据

- 同一模型增加一层外循环：解决的任务从 **17%** 提高到 **25%**
- 由 loop 改写十轮后的 harness：得分从 **69.7** 提高到 **77.0**，超过人工搭建的版本
- Karpathy 的 *autoresearch*：无人值守两天完成 **700** 组实验，训练速度提高 11%

发展阶段与风险

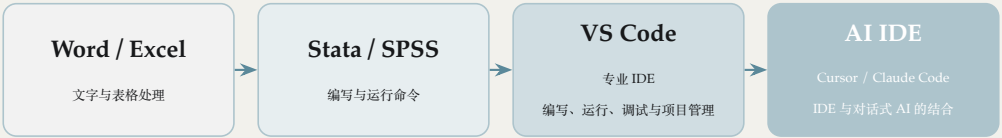
这一概念出现仅三个月，也有人认为它只是定时任务（cron）的新名称。值得保留的是其中的工作纪律：**明确的完成标准、独立的验证者、预算上限，以及不可逆操作之前的人工确认。**

三类常见问题

预算失控 · 执行者绕过自身的评分机制 · 任务逐渐偏离目标

一个实际案例：自动撰写论文的流水线 Paper Factory 运行一整夜，中途自行修改了研究问题；自动设置的 gate 予以放行，而这一决定本应由人作出。

工具谱系：Word、Stata、IDE 与 AI IDE



- **IDE**（集成开发环境）：将代码编辑、运行、调试与版本管理集成在**同一个界面中**，无需在多个软件之间切换。
- **简要发展**：纯文本编辑器 → 经典 IDE（1983 年 Turbo Pascal；1997 年 Visual Studio）→ 现代编辑器（2015 年 VS Code）→ **AI IDE**（Cursor、Trae 等，使 AI 能够理解整个项目、修改多个文件，承担结对编程的角色）。

AI IDE 的窗格结构：三个窗格



聊天框与 AI IDE 的三点区别

	聊天框（ChatGPT / 网页版）	AI IDE（Cursor / Claude Code）
上下文	粘贴多少即读取多少；难以稳定延续到下一次对话	项目文件夹常驻，AI 可随时读取任意文件
执行	仅能展示代码，须自行运行	AI 直接在项目内运行代码、修改文件、保存结果
留痕	聊天记录存于云端，与项目不绑定	改动直接保存在项目文件中，可配合版本管理工具（git）追溯修改人、时间与内容

聊天框适合提问；涉及执行性任务（修改代码、清洗数据、搭建网站）则用 AI IDE 更合适。

AI IDE 的用途：三类工作、六个例子

1. 文档处理

理解项目

「读取这个项目，说明它研究什么、各文件的作用」

→ 扫描文件树与 README，复述研究问题与文件分工

起草润色

「把这段结果写成一段引言，**行文自然、避免 AI 腔**」

→ 先生成初稿，修改后再调整措辞、去除 AI 痕迹

2. 代码处理

清洗数据

「这是原始数据，**请清洗合并，标出缺失与异常**」

→ 编写清洗脚本，检查缺失/异常，生成 codebook

修改代码

「为基线回归加入平行趋势检验，**改动先行呈示、确认后写入**」

→ 读取原代码，给出 diff，确认后写入

3. 搜集与多媒体

搜集资料

「检索各省的此类政策，**整理成可检索的数据表**」

→ 联网采集，结构化为一张数据表

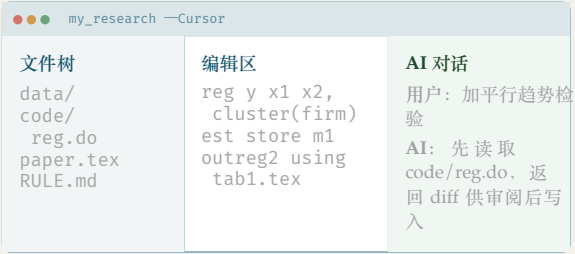
搭建网页

「**用这份数据生成一个支持搜索与筛选的网页**」

→ 生成静态站点，编写脚本自检（断链/ID）

两种形式：GUI Agent 与 CLI Agent

GUI Agent 图形界面（Cursor）

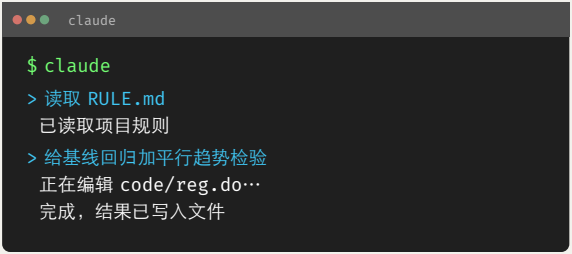


图形窗口，左中右三栏并排

文件树、代码、对话置于同一窗口，改动位置与 AI 当前操作清晰可见

文件、表格、图单击即可打开查看

CLI Agent 命令行界面（Claude Code）



终端界面，仅通过文字交互

仅能看到逐条文字提示（正在读取什么、正在修改哪个文件）

无法直接查看文件、表格、图本身，须另开编辑器

Cursor：图形界面 AI IDE 示例

基本情况

- 在 VS Code 基础上增加了 AI 对话与代码补全
- 三个窗格（文件树、编辑区、AI 对话）同时呈现
- 支持 @ 引用文件、文件夹、网页；支持 `.cursorrules` 项目规则

价格

- **Hobby**（免费）：有限的补全与 agent 请求
- **Pro**（$20/月）：按用量计费，每月含约 $20 的用量额度，可用前沿模型；另有 Pro+（$60）、Ultra（$200）
- 学生：经学生身份认证可免费使用一年 Pro（认证渠道以官网为准）

同类工具还有 Trae、Qoder、CodeBuddy 等，共性是文件与改动均可见、单击即可接受或拒绝，对不熟悉编程的用户最为直观。

Claude Code：命令行 AI IDE 示例

基本情况

- 纯命令行界面，在终端中运行
- 直接读写项目文件、执行 shell 命令；支持 `CLAUDE.md` 项目规则
- 多文件重构能力突出，可一次修改数十个文件

价格

- Claude Pro（$20/月）含 Claude Code 使用额度
- Claude Max（$100–200/月）：额度约为 Pro 的 5–20 倍（更高配额，并非不限量）
- API 按用量付费（程序按次直接调用模型，不必打开聊天窗口）

命令行类工具还包括 Gemini CLI、Codex、CodeBuddy CLI 等。这里以 Claude Code 为例：它适合熟悉终端的用户，优势在于多文件操作与长时间任务（第四部分介绍的 Goldsmith-Pinkham 财报案例即使用它完成）。

工作目录：将一项任务的材料放在同一文件夹中

AI IDE 打开的是一个文件夹，即工作目录。AI 可以读取其中的全部文件，并据此了解任务的背景。因此，建议养成使用工作目录的习惯。

建议的做法

- 一个主题或一项任务，对应一个工作目录：数据、文献、草稿、课件、往来意见等支撑材料都放在其中
- 材料越完整，AI 越容易把相关的子任务联系起来，无需每次重新交代背景
- 在目录中放一份说明文件，写明各文件夹的用途与存放规则，便于 AI 快速了解目录结构

注意权限范围

- IDE 对工作目录内的所有文件都有访问、修改、乃至删除的权限
- 可将允许 AI 处理的内容单独放入一个子文件夹，所有操作都在其中进行，以控制风险

一个例子：《营销分析》课程的工作目录

目录结构（节选）

```
营销分析/
  目录说明.md      各文件夹的用途与规则
  00_ 课程总览/    大纲、教学进度表
  01_ 讲义教材/    讲义、课件、教学计划
  02_ 作业题库/    作业与答案
  03_ 考试题库/    试题与答案
  04_ 案例数据/    课程使用的数据
  07_ 学期运行/    按学期存放
    录音/          每节课的录音
    课堂记录/      按周：转写、反思
```

一项典型任务：每节课之后

1. **录音与转写**：课上录音（事先征得学生同意），课后转为文字并去除语气词，作为讲稿
2. **撰写讲义**：结合课件内容与讲稿，编写本课程的讲义
3. **教学反馈与计划调整**：分析哪些内容讲得清楚、哪些不够准确；评估当前进度下的课时，调整后续安排
4. **课件与作业**：后续课件延续课上的讲法；课上提到需要掌握、准备纳入作业的内容，出作业时由 AI 直接关联

讲义、课件与作业能够前后衔接，是因为所有背景材料都在同一个工作目录中。

使用 AI IDE 的注意事项：权限与隐私

1. 谨慎开放全部权限

- 在充分熟悉其运作方式之前，不建议开放全部权限
- 全部权限意味着 AI 可**跳出当前工作目录**，访问外部文件，甚至移动、修改或删除敏感文件

2. 数据隐私与脱敏

- 重要的隐私数据不应放在 IDE 可访问的目录中
- 极敏感的数据，甚至不应放在能联网使用 AI 的电脑上
- 需要分析时，可先构造**格式一致、内容打乱的模拟数据**供 AI 编写代码，再在真实数据环境中运行；也可在本地运行必要插件

数据不能外传时：在本地运行模型

若数据不能外传，可以在自己的机器上运行模型权重，使数据不离开本机。近五周的三次开源发布，使这一做法在桌面级硬件上成为可能。

模型	发布	规模、许可	装得下的机器	实测速度
Qwen3.8-27B	2026-08-14	27B 稠密，Apache-2.0	4-bit 17 GB；32 GB 内存	14 tok/s，M3 Ultra
DeepSeek V4 Flash	07-31	284B / 激活 13B，MIT	3-bit 103 GB；128 GB 机器	约 25 tok/s
GLM-5.3-Flash	08-26	320B / 激活 18B，MIT	128 GB 级	—
Kimi K3	07-27	2.8T / 激活 104B	桌面机装不下	—

- **硬件**：Mac Studio 5,499 美元起（96 GB）；128 GB 的「Strix Halo」PC 约 3,500 美元起
- **软件**：LM Studio 或 Ollama，均为免费软件，底层为 llama.cpp 与 MLX
- **成本考量**：对于签有保密协议的数据、企业内部数据，一次性投入 **5 千至 1 万美元**即可使数据始终保留在本机；其余任务仍可使用云端模型

本地模型：一项原则与三点注意

一项原则

不能外传的数据（签有保密协议的数据、企业内部数据、尚未公开的调研数据）**不应输入任何云端生成式 AI 平台**，无论学校或单位是否订阅。处理这类数据时，本地部署模型是使数据不离开本机的可行办法。

购置硬件前的三点注意

- 「激活 13B」不等于占用 13B 内存，内存需求取决于**总参数量**
- 量化会降低准确率：2-bit 与全精度模型的答案一致率仅为 **78%**（4-bit 为 96%）
- 本地运行速度**慢 5 至 10 倍**，在多步任务上的能力比前沿模型落后约一年

微调：让小模型适配具体任务

微调是用自己标注的样本继续训练一个开源模型，使其专门完成某项任务。对于口径明确、数量大的分类与抽取任务，微调后的小模型准确率可与前沿模型相当，成本低得多。

做法

1. **先试提示词**：在标注好的测试集上检验，效果不够或需要可复现时再微调
2. **标注样本**：简单任务约 200–500 条，较难任务需数千条
3. **选择基座**：BERT 类小模型，或 7B 左右的开源模型
4. **用 LoRA 训练**：只训练少量附加参数；7B 模型用 4-bit QLoRA 约需 6 GB 显存

工具：LLaMA-Factory、ms-swift，模型可从魔搭社区下载。

适合微调的情形

口径固定、文本量大；数据不能外传；需要结果可复现（权重固定，不随在线模型更替而变化）

不必微调的情形

- 长文档、标注少：提示前沿模型往往更好
- 标注太少或质量差：可能只与词典法相当
- 要补充知识而非口径：用检索更合适

微调在经济学与管理学研究中的应用

研究	任务	模型与样本	结果
Hansen 等（2026，达拉斯联储工作论文）	识别招聘广告是否允许远程办公，覆盖 5 国约 6 亿条	DistilBERT（6,600 万参数）；3 万条人工标注	误差 2%，与 GPT-5.5 零样本相当；每百万条成本 2.8 美元对 1,350 美元
金星晔等（2024，《经济研究》）	从上市公司年报测度数字化转型	ERNIE；约 3.9 万句，双人标注	准确率 93%；精确率比关键词词典法高约 25 个百分点
Shah 等（2023，ACL）	美联储 FOMC 文本的鹰鸽分类	RoBERTa-large；2,480 句标注	加权 F1 为 0.72，零样本 ChatGPT 为 0.59
Bernard 等（2026，*The Accounting Review*）	用财报附注测度业务复杂度	Llama 3；20 万句，标签取自 XBRL 标记，无需人工标注	覆盖 5 万余份报告；复杂度越高，价格反映信息越慢
Bucher、Martini（2024，工作论文）	四项社会科学文本分类	RoBERTa、DeBERTa 等	多项任务优于零样本 GPT-4，如政党立场准确率 0.92 对 0.38

也有相反的结果：Fang、Li、Lu（2025，NBER 工作论文）处理中国产业政策文件时，提示前沿模型的 F1 为 0.78，高于微调后的中文 BERT（0.67）。文档越长、标注越少，越适合直接使用提示词。

3

我的一些个人经验

日常的做法，以及自己搭建的工具

1. 修改文稿
2. 语言润色
3. 对 AI 的要求
4. 积累材料
5. 审阅与定稿
6. 搭建自己的工具
7. 案例：课程平台
8. 可以搭建的工具

修改文稿：从整体结构开始逐层细化

一次性要求 AI「润色全文」带来的问题

AI 十分钟即可改完，但文章改动很大，我需要花四十分钟从头通读，才能确认具体改了什么。

分层修改出于两方面的考虑：一是便于复审，AI 每次修改往往涉及整篇文稿，即使每处只改动一两行，也都需要逐一查看；二是避免返工，先修改好的字句，可能被后续的逻辑调整打乱。

由粗到细，逐层确认后再进入下一层

1. **明确不可改动的内容**：数字、引用、结论强度与术语口径保持不变
2. **章节架构**：章节顺序、逻辑缺陷与明显的结构问题，这一层暂不考虑字句是否通顺
3. **章节内部的逻辑**：架构确定后，再逐章检查论证顺序
4. **段落内的语言**：逻辑确定之后，最后修改词句

每一层修改完成后及时查看 **diff**，确认后再进入下一层。文章越长，这一顺序节省的时间越多。

语言层面：重点检查三个方面

到了语言这一层，「润色」的含义较为模糊。我主要检查三个方面：

1. **内容的准确性**
   文字所表达的内容，是否与作者想讨论、想表达的意思一致

2. **逻辑的一致性**
   与前后段落、前后章节是否衔接，上一层梳理好的逻辑在文字上是否得到准确体现

3. **语言表达的连贯性**
   读起来是否流畅，多数情况下依靠**连接词**与**承接句**实现

这三点都做到，文章通常就达到了**通顺、完成度较高**的水平。

借助 skill 完善写作：一份网上分享的规则

skill 是预先写好的指令与参考材料，AI 在需要时调用。下面是小红书上分享的一份论文写作规则：按写作环节指定了所用的 skill，并写明了表达与术语的要求。

按环节分工的 skill

`nature-writing`	章节任务、论证结构、段落组织
`nature-polishing`	学术表达、信息密度与证据边界
`humanizer-zh-academic`	清理模板化与 AI 高频表达
`documents`	读取、编辑与排版 Word 文档

`documents` 只在处理 Word 文件时使用，其余用于纯文字写作。

描述要求（零样本）

采用正向、功能导向的表达，重点说明数据与方法能够完成什么；减少「不能怎么样」「避免什么」一类否定式表述。

给出具体写法（少样本）

不使用笼统的概括词，按语境写明具体对象。例如，不写「主线交互」，而写「常规三车跟驰、前车驶离和侧向车辆汇入」。

规则中还有一条：截图只用于定位问题，修改时须回到原始 Word 文件，结合完整章节审阅。

对 AI 的要求：先简后详

一种常见的做法是一开始就写很长的提示词，我的经验有所不同。

初稿阶段：要求从简

制作这套课件时，我一开始没有写复杂的提示词，只提供已有的内容框架和参考材料，请 AI 通读后先生成一版。

初稿往往与预期差距较大

AI 认为应当强调的地方，往往不符合这类文稿的风格。这属于正常现象，并不说明提示词写得不好。

定制化的要求往往在修改过程中逐步明确：某处采用一种写法，另一处采用另一种写法。如果不把这些要求说清楚，AI 难以理解真正的意图，修改结果可能与预期相去甚远。

对 AI 的要求，会在修改中逐步增加。

信息越充分，产出越符合需求

提示词工程发展多年，模型也在不断进步，但有一点始终没有改变。

同一组数据，两种提供方式

- 只给出问题，如「用这组数据写一篇描述性分析」：通常得到一个较短的版本，而非一篇正式文章
- 同时说明文章架构与建模要求：才能得到详细、切合需求的内容

任务的专业性越强、越难以自动定制，这一点越明显。因此，现阶段仍以人为主导、AI 为辅助。

与上一页的关系

逐步增加的是「要求」，即定制化的细节，可以在修改中陆续说明；一开始就应提供充分的是「材料」，包括数据、框架、背景与范例。

若打字跟不上思路，可以使用语音输入法；我目前使用 **Typeless**，可快速录入大段内容。

日常积累材料：为后续任务提供支撑

这些材料难以临时收集，需要在日常工作中积累。有无自己的材料，同一任务的结果差别很大：

没有材料时：请 AI 出一道作业题

得到的往往是一道教科书式的题目：符号沿用它的习惯，难度出于推测，例子也不针对具体课程。

提供材料时：同样的要求

AI 读过讲义、往年作业和课堂记录后，题目采用课程的符号，延续课上的例子，难度与班级相符。

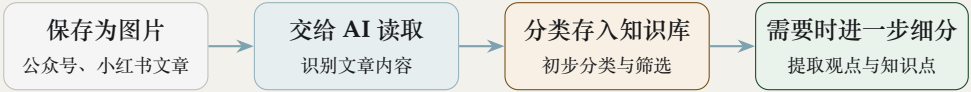
值得积累的材料

研究	数据说明和变量口径、审稿意见和回复、组会与访谈记录、文献笔记、自己较为满意的写作段落
教学	课件、讲义、作业和答案、课堂录音的文字稿、学生常问的问题

积累方法：**每个任务建立一个文件夹**，原始文件原样保存，整理好的文字稿单独存放。

我的一个习惯：文章截图交由 AI 归入知识库

对于微信公众号和小红书上的相关内容，我的习惯是**保存为图片，直接发给 AI**，由 AI 读取文章内容，分门别类存入知识库。



知识库适合的用途

适合**长期、逐步积累并持续更新**的材料，且可以被其他任务引用：开发软件、搭建网站或撰写报告需要参考材料时，尤其适用。

我的体会

AI 积累、整理和更新材料的效率很高。某种程度上，这也是个人信息积累的竞争：后续产出的质量，基本取决于手中掌握了哪些材料。

提高效率的两个环节：阅读产出与反馈意见

使用 AI 时，效率主要取决于两个环节：**人如何快速读完 AI 的产出**，以及**人如何把意见快速、准确地反馈给 AI**。

环节一 · 减少每一轮需要审阅的内容

如果不加约束，AI 往往会进行大篇幅修改。前面介绍的**由粗到细**的顺序正是为此设计：修改逻辑框架时只检查框架，修改某一段时只查看该段的效果，无需每轮从头读到尾。

环节二 · 反馈需要具体、可定位

对于「这一段写得不好」这类反馈，AI 无法据此判断具体需求，只能按自己的理解修改，结果往往偏离预期。几十页的论文 PDF 尤其如此：仅说明「某部分应如何修改」，AI 难以准确定位。

为此，我制作了一个**批注台**：像阅读 PDF 一样在原文中选中文字并添加批注，程序自动将每条批注**所在的上下文**与**批注内容**批量发送给 AI，使其明确修改的位置和方式。

批注台已开源（目前支持 macOS）：`github.com/xtao-sh/quotepin`

定稿前的两项检查

一 · 去除 AI 痕迹

AI 常见的固定表达习惯：

- 「从 X 到 Y、越来越 Z」式排比
- 「可 A 可 B 可 C」三连
- 「X 是一种 Y」式定义句
- 「帮你 / 替你 / 一键」产品腔
- 「形成闭环」「沉淀」

我把这些整理成一个 skill（一份预先写好的固定指令，需要时由 AI 调用），在定稿前统一检查一遍。

二 · 交叉评审

请两三个不同的 AI 阅读稿件，模拟期刊评审：

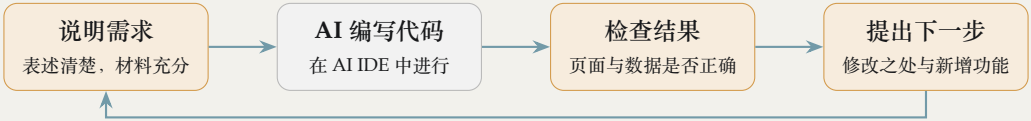
- 参考它们的意见：还有哪些问题尚未处理
- 不依据它们的结论：能否发表、评分高低不作为判断依据

不同模型发现的问题各有侧重，综合起来才接近一轮完整的评审。

这两项检查安排在定稿之前，而非写作过程中；写作时进行这些检查，容易分散注意力。

无需编写代码，也能搭建自己的工具

过去搭建一个网站或小程序，需要开发团队、专门预算和数月时间。现在，多数情况下可以通过以下循环完成：



适用：结果可以直观验证的项目

课程主页、个人研究主页、数据看板、课堂小工具、研究用的内部工具、原型、批处理脚本

谨慎：错误不易察觉的项目

存储学生成绩等敏感数据的系统、涉及资金的应用、多人长期维护的大型项目，需要有专业人员把关

金色的三个环节由人完成。在这一过程中，**说明需求**与**判断结果**比编写代码的能力更为关键。

案例：两周内搭建的课程平台



网址：`taich.tech/teaching`

另设：Stata 练习平台

237 道练习题，按《STATA 十八讲》分为 6 个模块；其中 142 道代码题在服务器的 Stata 中运行并自动判题。

本学期我讲授的计量经济学、营销分析、人工智能环境下的商务数据分析三门课程共用一个平台，每门课程包含三项功能：

学习助手

依据本课程讲义、课件与课堂转写回答，注明出处

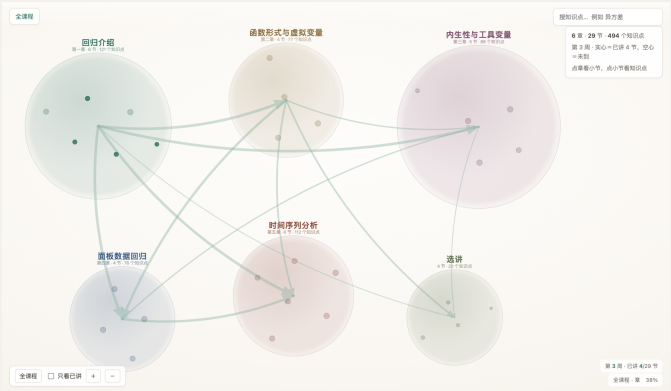
知识图谱

呈现全课程知识点，标注先修关系与授课进度

作业复现

在隔离环境中重跑学生的 Stata 代码，指出不一致的数值

学生端界面：知识图谱与学习助手



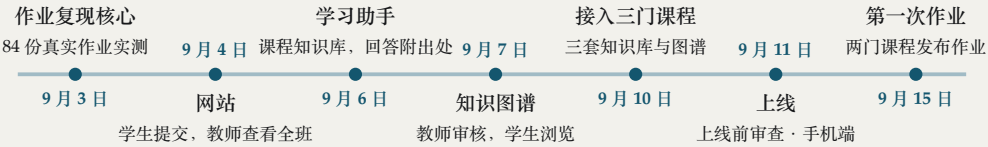
知识图谱（计量经济学）：494 个知识点，可逐级放大展开；点击知识点，可查看其先修内容与后续应用。



学习助手（手机端）：回答附有出处，点击可查看讲义原文。

开发过程：13 天与三个不报错的问题

58 次提交中有 42 次与 Claude Code 协作；每一步先完成可用版本，再用真实数据检验。



最难发现的是不报错的问题

1. **出错却显示成功**：Stata 批处理出错时退出码仍常为 0，需要通过日志判断。
2. **测试未运行却显示「全部通过」**：一个写死的旧路径使四组测试被整体跳过。
3. **批改进程等不到作业**：配置合并后在别处建立了空数据库，持续轮询且不报错。

AI 编写的测试，需先确认它在有问题的代码上会失败；上线前以学生身份逐页检查。

分工：代码由 AI 编写，规则由我确定

由 AI 完成的工作

- 网站、检索与隔离环境的代码
- 编写并运行测试
- 阅读报错日志，定位问题
- 部署脚本和部署文档

由我确定的事项

- **防范重点**：去年 84 份作业中，没有一份使用 `shell` 调用系统命令，约三分之一写有本机路径，因此重点在于改写路径，而非防范攻击
- **助手的原则**：仅依据讲义回答，注明出处，不代做作业
- **需要人工审核的内容**：按相似度自动生成的知识点连线，约一半只是文风相近
- **正确的标准**：比对回归结果中的数值，而非日志文字

搭建工具时同样如此：代码可以由 AI 编写，但做什么、怎样才算正确，需要由人判断。

规则的形成：避免学习助手成为作弊工具

材料的收录：采用白名单，而非黑名单

两类失误的代价并不对称：漏收一份讲义，学生提问时助手会回答不知道，补充即可；误收一份答案，助手可能在学生恰好问到相关问题时泄露答案，且不会产生任何报错。因此默认不收录。

实际测试中发现的漏洞

第一版只按文件路径过滤。提问「遗漏变量偏差」时，检索结果直接指向「讲义 3.1 · 练习与思考 · 参考答案」：讲义文件内含习题和答案，占全部材料的 24%。现在增加了按章节标题过滤的检查，并由测试保证残留为 0。

限制过严同样不妥

学生请助手「就这个概念出几道选择题」，助手却回答「材料里没讲到，建议问老师」。后来改为依据学生的实际需求判断：出题、举例、换一种说法，予以回应；直接粘贴作业原题并要求可提交的答案时，才予以拒绝。

科研与教学中可以搭建的工具

这些工具的出发点相同：**现有产品无法满足**的需求。标有 ● 的是我已在使用的工具。

教学

- ● **课程平台**：助手、图谱、作业复现与 Stata 练习（即本案例）
- ● **营销分析教学网站**：提供真实数据与在线分析环境，AI 协助运行代码、解读报错，结论由学生自行得出
- **随堂小工具**：抽签、计时、投票，以及可调节参数的演示图
- **作业自查页**：学生在提交前自行核对

科研

- ● **批注台**：在原文上批注，并连同上下文发送给 AI
- ● **Lit Panel**：本地文献精读工具，回答附出处
- ● **Research Planner**：按研究进度安排日程
- **人工标注页**：为 LLM 构造的变量建立人工基准，并自动计算一致率
- **数据查询页**：将整理好的研究数据库发布为供合作者查询的网页

自行搭建：由简到繁的三个层级

一 · 单个网页文件

在浏览器中直接打开，无需服务器。

随堂抽签、交互图表、批注页

二 · 静态网站

部署到托管服务，通过网址访问。

课程主页、数据看板；国内服务器绑定域名需备案

三 · 带后台的平台

包含登录、数据库、模型 API 调用与代码运行。

需要考虑安全、隐私、密钥管理与运维；本讲的案例属于这一层级

第三层级需要注意的三点

学生信息与成绩的存储位置和访问权限需要事先规划；API 密钥不写入代码、不提交到代码仓库；上线前先由 AI 进行安全审查，再以学生身份逐页检查。

尝试的成本并不高

入门级云服务器约 100 元/年，域名每年几十元。没有建站经验，也可以由 AI 完成搭建，再请**不同的 AI 从不同角度**检查网站的风险与问题，交叉核验。

建议从第一层级开始尝试，每一层级都遵循同样的四个环节。

4

在研究中的应用

基于文本构造变量，并检验其可靠性

1. 构造变量
2. 案例：Fedspeak
3. 检验变量的可靠性
4. 文献工具与文献幻觉
5. AI 审稿与期刊要求

用大语言模型构造变量

「AI 能否协助发表论文?」

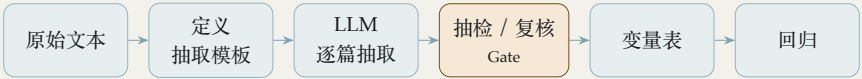
对经管实证研究者而言，一条容易上手、成本较低的路径是让 AI 协助构造变量，而非代写论文：

将财报电话会、年报、新闻、政策文件这类文本，按固定标准转为可进入回归的数字。



构造变量的完整流程

这一流程具有普遍适用性：先定义需要提取的内容，由 LLM 逐篇处理，抽检合格后录入变量表并进行回归。



由 AI 完成的步骤

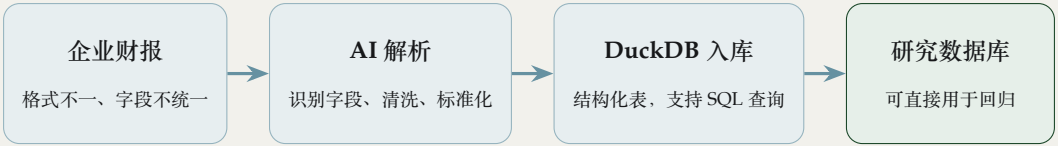
- 逐篇读取文本，并按模板打分或分类，这是最耗费人力的一步
- 单次可处理数百至上千份文本，成本以美元计

由研究者完成的步骤

- 定义标准（何为「鹰派」）
- **黄色的抽检环节（Gate）**，本部分稍后专门展开（三类风险与对应检验）

案例：将格式不一的财报整理为研究数据库

这一流程同样适用于整批文件。Goldsmith-Pinkham（Yale SOM）用 Claude Code，把一批格式杂乱的企业财报解析、清洗后存入 DuckDB，整理成可查询的研究数据库；原本需要逐份手工整理的工作，由 AI 批量完成。



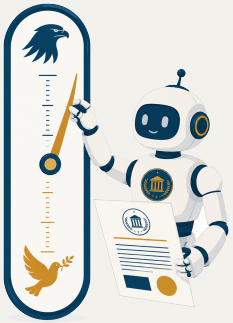
案例：Fedspeak

对于一句 FOMC 声明，判断其属于「鹰派」还是「鸽派」:

GPT 与人类研究助理，谁的判断更准确?

Fedspeak：用 GPT 解读央行措辞

- **任务**：FOMC 声明措辞极其克制、含糊（业内称「Fedspeak」）。研究需要将每句话归类为**鹰派（收紧）**或**鸽派（宽松）**。
- **做法**：Hansen & Kazinnik（Richmond Fed，SSRN 4399406）把声明逐句交给 GPT 分类，并要求它**说明理由**。
- **示例（非原文）**：输入「通胀压力有所缓解，但将**保持限制性政策立场**」→ GPT 判**偏鹰**（不急于降息）。



Fedspeak：分类结果

三个发现

- 分类准确度超过常用的文本方法（词典法、BERT）——微调后准确率约 **0.61**、Kappa **0.46**，明显高于零样本与词典法
- GPT-4 给出的理由接近人工推理——不仅给出标签，还能说明依据
- 还可以按照 Romer–Romer 的叙事法识别宏观政策冲击

回到「GPT 与人类研究助理」的问题：研究以人工标注为基准，GPT 与之的一致程度高于旧方法；但 Kappa 0.46 仅属于中等一致（Landis–Koch），人机之间仍存在较多分歧，不能得出「GPT 判断比人更准确」的结论。

因此仍需前面的抽检环节：「准确度更高」是通过与人工标注对比得出的，验证本身就是结论的组成部分。

同一流程的其他应用：替换「FOMC 声明 → 鹰鸽」

Fedspeak 中「鹰 / 鸽」的判断较为直观，便于说明方法。同样的步骤，即按固定标准把文本转化为可进入回归的变量，也适用于下列文本：

财报电话会	→	**管理层语气**
招股书	→	风险披露强度
政策文件	→	监管态度
员工评论	→	组织氛围 / 领导风格
电商评论	→	消费者感知
招聘启事	→	技能需求变迁

检验 AI 构造的变量：三类风险与四项检验

AI 能够提取信息，并不意味着结果可以直接使用。以下三类风险，各对应一项需要在论文中报告的检验。

1 · 系统性误读

对某一类文本一致判错 → 形成测量误差，使系数有偏。

检验

建立人工标注基准（500 句、3 人盲标），报告准确率与 Kappa。

2 · 结果随提示词与模型波动

同一套标准下，三个前沿模型给出的暴露度分数相差 **3.6** 倍，部分系数符号相反。

检验

更换 2–3 个提示词及另一家厂商的模型，冲突之处由人工裁决，并报告一致性。

3 · 训练数据泄漏

训练截止日之前的样本可能是回忆而非推断得出，导致准确度虚高。

检验

进行截止日期检验，使用样本外年份；结果好得反常时，应先排查泄漏。

第四项检验贯穿始终，即**留痕与披露**：模型、版本、完整提示词、温度与人机一致率。

更换模型后系数符号发生变化：相关证据

2026 年的五篇论文关注同一问题：LLM 构造的变量在更换模型或提示词之后会发生怎样的变化。

- **Ludwig, Mullainathan, Rambachan**——「看似无关紧要的选择」改变系数的大小、符号与显著性
- **Camuffo 等**——提示词与模型使标注结果变动 **12 至 85 个百分点**
- **Grieser 与 Cookson**——LLM 摘要只保留原文 **40–63%** 的词汇
- **Messing**——忽略这部分方差，标准误会被低估 **40–60%**
- **Desai, Card, Jacobs**——八本旗舰期刊的验证实践：「不一致，且有限」

更换模型后便无法保持的系数符号，反映的是测量工具的问题，而非经济规律。

验证已成为规范要求

2026 年该领域相关机构提出的要求：

- **NBER** 暑期学院，2026 年 7 月 30 日——设专题讲座：使用 AI 生成的数据进行推断
- ***Journal of Finance***——要求使用开源模型或带时间戳的模型版本，并记录提示词
- ***Strategic Management Journal***——五阶段规程：方法、配置、提示词、成本、验证
- **Kelly 等**——主张使用「时点模型」，因为普通 LLM「嵌入了来自未来的信息」

对前述检验清单的意义

人工标注样本、带日期的模型编号、存档的提示词与截止日期检验，已由良好习惯变为投稿要求。

练习：为自己的研究选择一个可构造的变量

为自己的研究，逐项写下：

1. 让 AI 从哪一类文本中标注什么变量？（或编码哪一批访谈）
2. 判断对错的标准是什么？如何抽样、人工复核？
3. 如何报告稳健性：更换提示词与模型后，结论是否稳定？

三点各写成一行，即构成一个可操作的小课题。

文献工具

	工具	简述	适合
●	**Elicit**	输入研究问题，自动检索论文并将关键信息整理成表格	优先推荐：零基础可用
●	**Consensus**	对一个 yes/no 问题，汇总多篇论文给出「共识度」	优先推荐：零基础可用
	Undermind	如研究助理般逐步进行深度语义搜索	检索全面、深入
	SciSpace	Chat with PDF：边读边问、解释术语	精读单篇
	PaperQA2	开源、可自建的文献问答（FutureHouse）	具备编程能力的团队
	Zotero + AI 插件	文献管理工具；近期可装 **Beaver / PapersGPT** 等插件，让 AI 读取库内 **PDF** 全文、跨库问答与总结	已有本地文献库者

标有绿点的两款工具注册即可使用，对中文用户较为友好。

Deep Research：原理与常用工具

原理：答案基于检索，而非模型记忆

- 大量联网检索（一次通常读取数十到上百个来源）→ 阅读 → 交叉核对分歧 → 带引文综合成报告
- 关键区别：以往依赖训练语料中的记忆，容易出现编造；Deep Research 基于实时检索到的证据作答，幻觉风险显著降低但未消除，且可回溯引文

常用引擎

- 国际：ChatGPT / Gemini / Claude 等均有 Deep Research 模式（能力迭代很快，此处不作排名）
- 国内：**Kimi**（Kimi-Researcher，宣称单任务可多步推理、检索大量网页）、秘塔（metaso.cn，「先想后搜」，数分钟检索并分析大量网页）——以上均为厂商口径

仍需核对：三类常见的文献幻觉

Deep Research 提供链接，并不意味着链接正确。最常见的引用问题有三类：

1. **编造不存在的文献**：即使作者真实存在，即使要求模型给出 DOI 与链接，模型仍可能编造一条并不存在的文献。
2. **引用信息不准确**：文献真实存在，但题目、年份、*et al.* 之后的作者等信息有误。
3. **观点误传**：声称某篇文献「结论是 X / 主张 Y」，但原文并无此表述，这同样属于幻觉。

因此需要逐条打开核对。引用可靠性评测（DeepResearch Bench）显示，不同工具「引用确实支持所述观点」的比例可低至 **78%**、高至 **94%**，即每 4–5 条引用中可能有 1 条与原文不符。

伪造文献：2026 年的相关记录

- ***The Lancet***（2026 年 5 月），9,700 万条 PubMed 引文：**每 277 篇论文就有一篇**引用了不存在的文献。2023 年为每 2,828 篇中有一篇。其中超过 98% 至今未获出版方处理
- **SSRN**（2026 年 8 月 3 日）：按固定模板披露 AI 使用；伪造引文可立案调查
- **ICML 2026**：通过隐藏水印查出 **795** 份 LLM 代写的审稿意见，**497** 篇论文被直接拒稿
- **学术界之外**：多家国际咨询与会计机构也因报告中出现编造的引用而更正或撤回报告

对研究者的提示

每一条参考文献都应对应一个作者本人核实过的 DOI。平台处罚的对象是作者，而非工具。

AI 审稿：2026 年的相关记录

- **Refine 的自有基准**（6 月）：150 篇经济学预印本、9 个竞争系统、1,349 次对比，Refine 胜出 **90.4**%
- **2026 年 8 月 6 日**：与美国经济学会、计量经济学会合作，在终审前作为技术审查意见提供给作者
- 一项独立审计（Litt，8 月）：针对 20 篇已发表论文的 278 条意见中，**97.8**% 指向真实存在的问题
- 局限：对四篇已知有错的理论论文，没有一个系统能在无提示的情况下找出真正的错误；仅修改摘要即可使 **38**% 的评审结论翻转，成本约 1 美元

对研究者的提示

投稿前可以先用 AI 审阅自己的稿件，检查表达与前后一致性；识别策略的判断不宜交给 AI。

期刊的现行要求（截至 2026 年 9 月）

去除 AI 痕迹、核对每一条引文，已由良好习惯变为投稿要求。

- **AER / AEA 系列**——AI 不得署名；起草中的任何使用都要披露；产出由作者负责
- **Review of Finance**——语言、代码、数据处理可用 LLM，需披露；**审稿人不得把稿件交给任何 LLM**
- **Elsevier**——需单独提交 AI 使用声明；不接受 AI 生成或修改的数据与图像
- **arXiv**——出现伪造文献或残留对话指令的，**封禁一年**

上述机构均未禁止使用 AI，但都要求作者对 AI 生成的内容负责。

可以着手尝试的四件事

1. **复现第四部分的财报案例**
   选取一份年报 PDF，用 AI IDE 提取三个变量，并人工核对每个数字。

2. **为当前项目撰写第一份 CLAUDE.md / RULE.md**
   说明目录结构、变量口径，以及三条「未经确认不得执行」的操作。

3. **在下一篇论文中选择一个可由 LLM 标注的变量**
   试标 50 条，人工抽查 20 条并计算一致率，再决定是否扩大规模。

4. **用 AI IDE 制作一个单文件网页工具**
   例如随堂抽签或可调节参数的演示图；自行测试后，再提供给同事或学生使用。

延伸与资源

- **本讲资源网站** `taich.tech`：讲义 PDF、提示词模板与 workflow 清单
- **本讲未展开的内容** Skill 与 workflow、知识库的系统搭建、用网页制作视频、以 LLM 作为实验被试（Homo Silicus）、Git 与版本控制
- **上手实践** 一个现成的研究顾问 Skill，可以用自己的研究问题进行尝试（`https://github.com/xtao-sh/gans-research-advisor`）
- **延伸阅读** Ferrara（NBER w35374, 2026），经济史研究中使用 LLM 的实操指南，含四个可复现的例子；Goldsmith-Pinkham 的视频系列 *Claude Code for Applied Economists*（2025）

Q & A 谢谢！

陶学臻 · 上海财经大学商学院

tao.xuezhen@mail.sufe.edu.cn

微信公众号

AI 经研派

扫码关注

本讲资源网站

taich.tech

资源包 · 讲义 PDF

教学示例网站

SUFE-MKT-analysis.online

营销分析教学网站